

LLM 后训练之 Judge / Verifier

深度研究与工程手册

从人类偏好、LLM-as-a-Judge、Reward Model 到 Rule / Code / Formal Verifier、PRM 与 Scalable Oversight

Judge / Verifier = Objective to Training-Signal Compiler



Core risks: spec mismatch, weak verifiability, bias, OOD failure, reward hacking, over-optimization

研究日期：2026-08-13

主题：LLM Post-Training / Reward Engineering / Verification / Scalable Oversight

0. 执行摘要：Judge / Verifier 是后训练的“训练信号编译器”

核心定义： Judge / Verifier 的任务不是“给回答打一个分”，而是把 Objective / Behavior Spec 转换成可用于数据筛选、偏好对、RL reward、test-time search 与模型验收的可靠训练信号。

后训练真正困难的地方往往不是优化器，而是训练信号是否真的代表目标。一个错误、可被利用或分布外失效的 Judge，会让更强的优化算法更快地把模型推向错误方向。

工程上应把 Judge 与 Verifier 区分：Judge 更擅长开放式、主观、语义复杂的“好不好”；Verifier 更擅长有外部事实、规则、执行环境或形式系统支持的“对不对”。Reward Model、Critic、PRM、RBR 等处于两者之间的连续谱。

成熟系统不依赖单一裁判，而是采用分层信号：确定性 verifier 处理能验证的部分；LLM Judge / RM 处理语义部分；Critic 给人类提供证据；高价值或高风险切片由独立 human/gold audit 兜底。

问题	工程答案
什么能被可靠判断？	先做 Verifiability Decomposition：把 correctness、format、safety、style、task success 分开。
谁来判断？	选择 Human / LLM Judge / RM / ORM / PRM / rule / code / formal / environment / ensemble。
怎么进入训练？	Filter、pair construction、scalar reward、dense step reward、rejection sampling、best-of-N、eval。
怎么防止骗裁判？	独立 holdout、OOD test、reward cap/clip、组合奖励、对抗样本、judge rotation、human audit。
怎么衡量 Judge 自己？	pairwise accuracy、calibration、slice FPR/FNR、stability、OOD、hack gap、cost/latency。

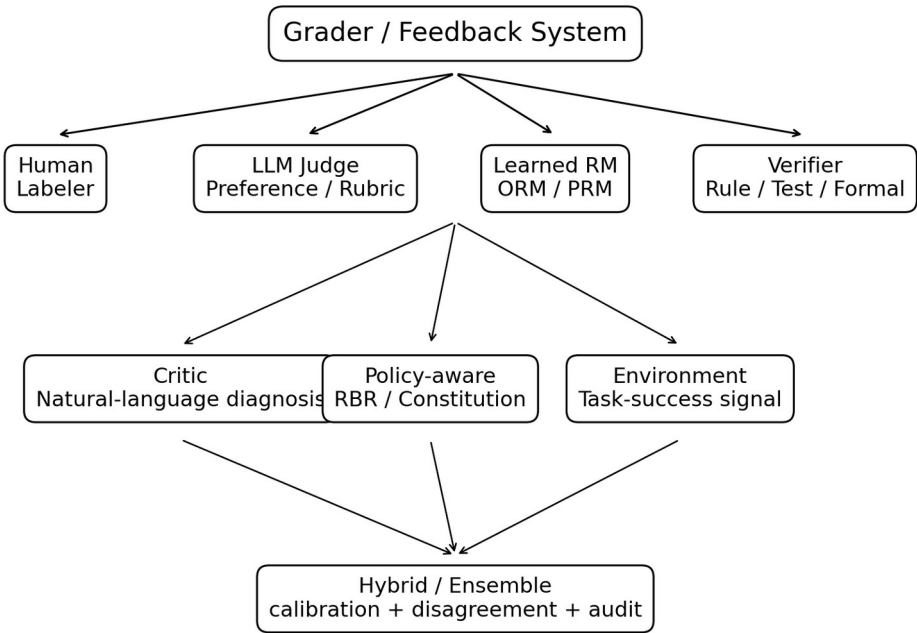


图 0-1 Judge / Verifier 体系不是单点模型，而是一组互补的监督组件。

1. 为什么 Judge / Verifier 是 Post-Training 的核心瓶颈

后训练可以被抽象为：Policy 生成行为，评价系统产生训练信号，优化器重新分配概率质量。因此，评价系统决定了“梯度往哪里走”。

Prompt → Policy → Response / Trajectory → Judge / Verifier → Reward / Preference → Update

层	如果判断系统出错	结果
Data Filtering	好样本被丢、坏样本被收	SFT 学到错误示范
Preference	chosen / rejected 颠倒	DPO 学错偏好方向
RL	proxy reward 可被利用	Reward Hacking / 过优化
Search	错误候选被 rerank 到第一	test-time compute 反而放大错误
Eval	评价指标本身偏置	“指标变好、产品变差”

Know-Why: 优化能力越强，Judge / Verifier 的质量越重要。弱优化器会缓慢追逐错误代理目标；强 RL / 大规模 search 会更快地发现并利用代理目标的漏洞。

1.1 从 Human Preference 到 Scalable Oversight

- 传统 RLHF：人类比较回答，Reward Model 学习人类偏好，再作为 RL 的代理目标。
- LLM-as-a-Judge：利用强模型在开放式任务上替代部分人工评价，成本低、可解释，但存在位置、长度、自偏好与推理能力限制。
- Critic / AI-assisted oversight：不是让 AI 直接替代人，而是让 AI 提供问题定位与证据，提升人类判断能力。
- Rule / executable / formal verifier：把“是否正确”外包给测试、编译器、数学规则、证明器或环境状态，形成更可靠、可扩展的 RLVR 信号。
- Process supervision：从“最终结果正确”进一步检查“中间过程是否可靠”，但标注与泛化难度更高。

2. Judge、Verifier、Reward Model、Critic：概念边界

概念	输出形态	最适合的问题	主要风险
Judge	分数 / 偏好 / 等级	开放式质量、风格、helpfulness、综合偏好	position / verbosity / self-preference / prompt sensitivity
Verifier	0/1、结构化错误、通过/失败	答案、代码、格式、环境目标、形式证明	覆盖不足；“通过测试≠整体正确”
Reward Model (RM)	标量 reward 或 pairwise score	大规模偏好代理、RLHF	OOD、过优化、reward hacking
ORM	整条响应/最终结果分数	Outcome correctness	稀疏 credit；正确结果可能来自坏过程
PRM	步骤级 reward	多步 reasoning / process supervision	step label 贵；跨难度泛化差
Critic	自然语言诊断 + evidence	辅助人类发现隐藏错误	hallucinated critique / 漏检
RBR / Policy-aware grader	规则 proposition + score	安全、格式、明确行为约束	规则覆盖、grader bias、规范维护
Environment grader	任务状态/成功率/成本	Agent、工具使用、软件工程	环境噪声、credit assignment

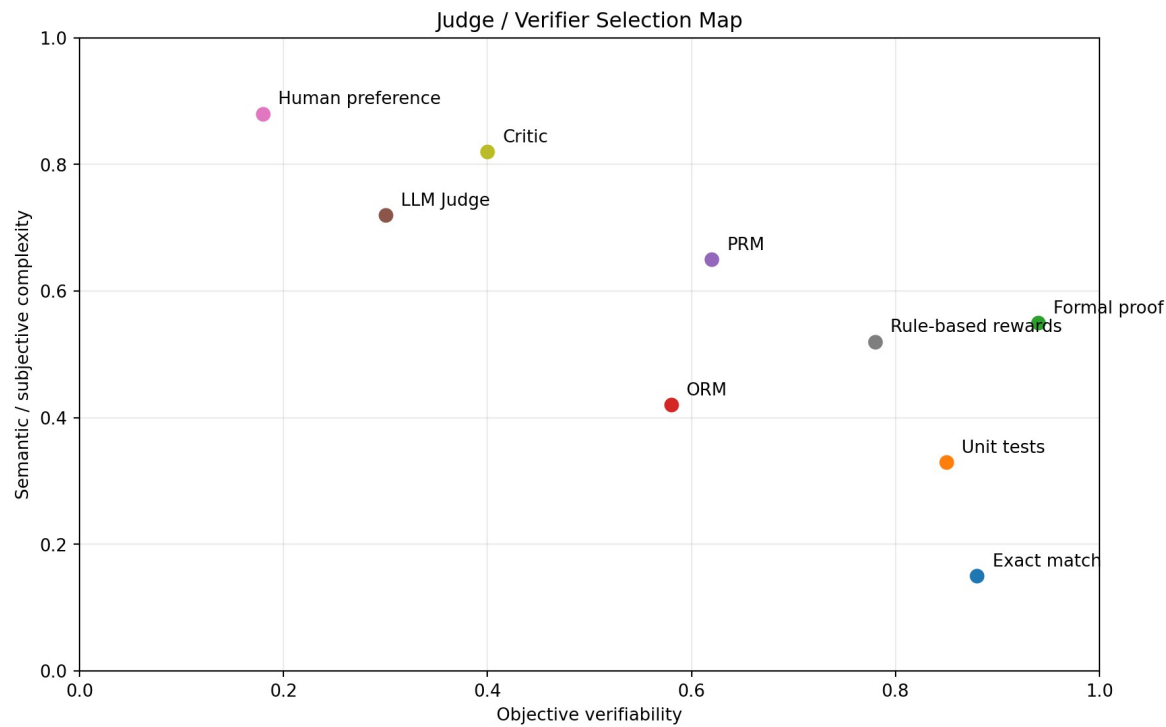
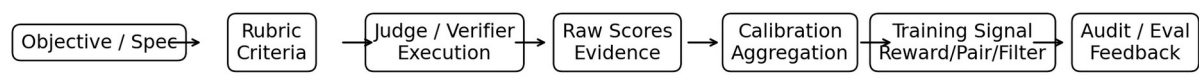


图 2-1 选择方法时，先看任务是否存在可外部验证的事实，而不是先决定用哪种模型。

最重要的区分： Judge 回答“更像一个好答案吗？”，Verifier 回答“是否满足可检查的事实/规则/环境条件？”。能用 verifier 的部分应尽量 verifier 化，把主观 Judge 留给真正不可形式化的剩余部分。

3. Judge / Verifier 的完整生产架构

Judge / Verifier = Objective to Training-Signal Compiler



Core risks: spec mismatch, weak verifiability, bias, OOD failure, reward hacking, over-optimization

图 3-1 Objective-to-Signal Compiler

模块	输入	输出	核心 Know-How
Spec Registry	Objective / Model Spec	版本化原则与优先级	任何 grader 都必须绑定 spec_version
Rubric Compiler	Spec + task taxonomy	可判定 criteria	把综合“好”拆成 correctness / relevance / style / safety 等
Grader Router	task metadata	grader plan	按任务选择 deterministic / learned / human
Evidence Layer	response + refs + env	test log / citation / trace	让 judgment 有证据，不只给分
Calibration	raw scores	calibrated scores	阈值、temperature scaling、slice calibration
Aggregator	多信号	reward / verdict	组合奖励、优先级、hard constraints
Audit	sample + disagreement	gold labels / error taxonomy	持续发现 grader 漏洞与 drift

推荐的组合目标： $R_{total} = w_c R_{correct} + w_t R_{task} + w_s R_{style} + w_{safe} R_{safe} - \lambda_{cost} C - \lambda_{hack} H$

但注意：并非所有维度都适合线性加权。对硬约束（例如 schema validity、关键安全边界、unit test）更适合 gating / lexicographic priority，而不是让“写作风格高分”抵消“答案错误”。

4. LLM-as-a-Judge：何时有效，何时危险

MT-Bench / Chatbot Arena 工作证明强 LLM Judge 在开放式对话上可以与人类偏好达到较高一致性，但同时明确指出位置、冗长度、自增强/自偏好和有限推理能力等偏差。因此，LLM-as-a-Judge 应被视为“可扩展代理监督”，不是无条件的 ground truth。

Judge 形式	优点	缺点	适用
Pointwise	易并行、可绝对评分	尺度漂移、跨 prompt 不可比	单响应质量、rubric score
Pairwise	比较更容易、适合 preference	position bias、无法表达都差	DPO/RM pair
Listwise	可排序多个候选	上下文长、排序不稳定	rejection sampling / candidate rank
Reference-aware	可检查事实/关键点	依赖 reference 质量	QA、摘要、标准答案
Reference-free	覆盖开放任务	更主观、更受风格影响	写作、对话、通用助手
Rubric-guided	可解释、可定制	rubric 本身可能遗漏	产品行为、细粒度质量
Reasoning Judge	复杂判断能力更强	成本与自信幻觉更高	难推理、process critique

4.1 Judge Bias Checklist

偏差	表现	检测方法	缓解
Position bias	同一 pair 换序结果变化	swap test	双向评分 + tie/一致性门
Verbosity bias	更长回答获得更高分	length-controlled pairs	criteria decomposition + 长度归一
Self-preference	偏爱自身/同系列风格	cross-family judge	judge ensemble / independent model
Style bias	格式漂亮掩盖事实错误	fact-conflict adversarial set	correctness-first gating
Authority bias	引用/自信语气导致高分	伪引用对抗样本	外部 evidence verifier
Prompt sensitivity	rubric 措辞改变排序	prompt perturbation	固定模板 + versioning
Capability ceiling	Judge 自己解不出题	oracle/solver comparison	强 verifier / tool-assisted judge

5. Reward Model：把人类/AI 偏好压缩成可优化代理

Reward Model 的本质是学习一个代理效用函数 $R\phi(x,y)$ 。它可以用于 pairwise ranking、best-of-N、rejection sampling 和 RL。问题在于：一旦 policy 直接优化 $R\phi$ ，就会主动寻找 $R\phi$ 与真实目标之间的缝隙。

$$L_{RM} = - \log \sigma(R(x, y_{chosen}) - R(x, y_{rejected}))$$

RM 设计维度	关键选择	工程影响
训练标签	Human / AI / hybrid	决定 value source
数据分布	on-policy / off-policy / hard negatives	决定对新 policy 的适用性
输出	scalar / multi-head / pairwise	可解释性与组合方式
目标	helpfulness / safety / correctness / style	不要把冲突目标无脑合成一个分数
更新频率	frozen / periodic refresh	旧 RM 会随 policy 提升而 stale
独立性	与 policy 同源 / 异源	同源越高，相关偏差风险越大

RewardBench 的意义： Reward Model 本身也需要 benchmark。RewardBench 专门用 chosen/rejected 对测试 chat、reasoning、safety 以及含细微可验证错误的样本，提醒我们不能只看 RM 训练 loss。

6. Outcome Verifier / ORM：先把“最终正确”做可靠

- Exact match / normalized match：适合答案空间规范、可标准化的题。
- Parser + schema：验证 JSON、函数调用、结构化输出。
- Execution：代码编译、unit tests、SQL 执行、模拟器、工具环境。
- Ground-truth comparison：有标准答案或可计算目标。
- Learned ORM：最终结果无法直接机械验证时，用模型学习 outcome correctness。

DeepSeek-R1-Zero 案例：其公开技术报告明确采用 rule-based accuracy reward 与 format reward；数学答案用指定格式支持可靠规则验证，代码可通过 compiler/test cases 反馈，并明确表示未使用 neural outcome/process RM，原因之一是大规模 RL 中的 reward hacking 风险和维护成本。

优点	局限
信号稳定、低成本、可扩展	只能验证“结果”，无法保证过程可信
适合大规模 RLVR	测试覆盖不全会诱导“只过测试”
对 policy shift 较稳	开放式任务难以构造 deterministic oracle
易做独立复现	答案格式、等价表达需规范化

7. Process Reward Model (PRM)：监督“怎么得到答案”

Process supervision 在每个推理步骤上提供反馈。OpenAI 的 Let’s Verify Step by Step 在 MATH 上发现 process supervision 显著优于 outcome supervision，并公开 PRM800K；Uesato 等工作也显示 process-based supervision 能显著降低最终答案正确但推理过程错误的比例。

维度	ORM / Outcome	PRM / Process
信号位置	最终结果	每一步/局部过程
Label 成本	低	高
Credit assignment	弱	强
可解释性	较低	较高
Reward hacking	可通过坏过程碰巧答对	可能通过堆叠“看似正确步骤”刷分
泛化	相对容易	难度分布外泛化是明显问题
适用	可验证结果任务	复杂 reasoning / 长轨迹

ProcessBench 的警告：在更难的竞赛/奥数推理上，已有 PRM 往往不能很好泛化，甚至落后于直接 prompt 的 critic 模型。这意味着 PRM 不是“训练一次永久使用”的通用 verifier，必须做难度与领域切片验证。

ThinkPRM 的方向：2025 年 ThinkPRM 探索“生成式 PRM”：让 verifier 先生成验证 CoT 再逐步判断，以极少量 process labels 获得更强 OOD 与 test-time verification 能力，体现从 discriminative scorer 向 reasoning verifier 的演进。

8. Critic：把“分数”升级为“可定位的错误证据”

Critic 的输出不是单一 scalar，而是自然语言诊断、错误位置、证据与修复建议。它特别适合 scalable oversight：人类面对强模型输出时，真正缺少的常常不是最终投票能力，而是“我应该检查哪里”。

模式	输出	价值
Self-critique	模型批评自己的回答	低成本发现明显错误
Independent critic	独立模型找缺陷	减少生成与评价相关性
Human + Critic	AI 提示问题，人类裁决	提高人类监督能力
Critic + RM	自然语言 critique 再由 RM 评分	兼顾证据与标量 reward
Critic search	对 critique 做 best-of-N / RM search	调节 precision-recall

CriticGPT 案例：OpenAI 报告 Human+CriticGPT 在代码审查中优于无辅助人类的比例超过 60%；同时，model-only critic 仍会 hallucinate bugs，因此 human-machine team 是更稳妥的监督结构。

9. Rule-Based Rewards / Policy-aware Grader：把规范“编译”为可执行评判

当目标可以被拆成明确 proposition（例如是否遵从、是否包含不允许内容、是否提及政策、是否满足格式），可以使用规则驱动或 specification-aware 的 grader。其优势是可解释、易更新，并可与 learned helpfulness RM 组合。

组件	示例	Know-How
Proposition	complies / helpful / format-valid	必须原子化，避免一个规则包含多个隐含条件
Rule set	允许 / 不允许 / 默认行为	版本化 + 优先级
Grader	固定 LLM / classifier / rule engine	对每个 proposition 独立判定
Aggregator	linear / lexicographic / gating	硬约束不要被软分数抵消
Calibration set	已知理想类型的 prompt/completion	验证阈值和权重
Policy update	PPO/GRPO / rejection sampling	reward 变化要做回归

OpenAI RBR 案例：Rule-Based Rewards 通过显式规则 proposition 评价响应，并与 helpful-only reward 等结合进入 RL；官方强调它适合清晰规则，对高主观性任务仍应与人类反馈等信号组合。

Deliberative Alignment 延伸：更进一步的做法，是让 reward model 本身访问规范，并让 policy 在推理中显式应用规范；这把“规范只生成标签”升级为“规范进入模型与 evaluator 的 reasoning context”。

10. Code / Tool / Agent Verifier：从“答案评分”进入“环境闭环”

Agent 能力	Verifier	奖励例子
File editing	diff + unit test	tests passed / patch validity
Coding	compiler + tests + static analyzer	compile=1, tests ratio, regression penalty
Tool use	schema + tool execution	valid args, successful call, task state
Web/RAG	citation checker + source evidence	citation entailment / groundedness
SQL	database execution	result equivalence
Computer use	UI state / environment snapshot	goal state reached
Long-horizon agent	environment + milestone checks	final success + intermediate milestones + cost

10.1 Agent Verifier 的额外困难

- Credit assignment：最终失败不意味着所有中间动作都错。
- Partial observability：grader 未必能看到真实内部环境状态。
- Non-determinism：网络、API、UI、测试环境会引入噪声。
- Shortcut exploitation：agent 可能修改测试、绕过目标、伪造完成状态。
- Cost-aware success：只看 task success 会鼓励无限工具调用与超长轨迹。
- State reset：训练环境必须可重复，否则 reward 不可比较。

Agent reward 设计：优先使用“环境状态 + 独立测试 + 轨迹约束”的组合，而不是仅让另一个 LLM 阅读 transcript 后凭感觉打分。

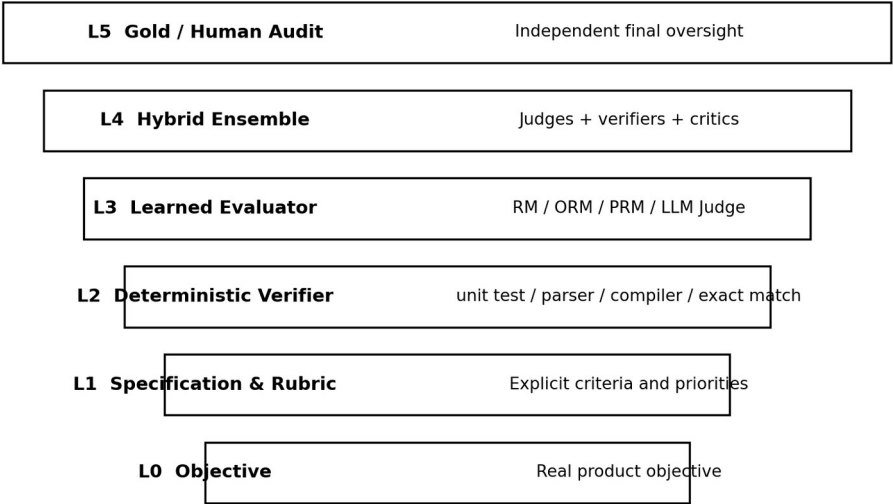
11. Formal Verifier：最高置信度，但覆盖最窄

形式验证把自然语言问题映射到可由 proof assistant / solver / symbolic system 检查的对象。它是最接近“机器可证明正确”的 verifier，但需要形式化接口和严格表示。

类型	典型工具/机制	优势	局限
Formal proof	Lean / Coq / Isabelle 等	机器检查 proof	生成形式证明难
SMT / SAT	约束求解	精确、可复现	只覆盖可形式化问题
Symbolic math	CAS / algebra checker	等价式验证	自然语言步骤映射难
Structured intermediary	自然语言→结构化步骤→formal check	兼顾表达与验证	需要转换层可靠

发展方向：2026 年已有研究探索“结构化中间表示 + formal prover”逐步验证自然语言推理：并非要求模型直接生成完整形式证明，而是把中间步骤转成机器可检验对象。

12. Human + AI：人类监督不是被替代，而是被“放大”



Higher layers cost more but are more independent; lower layers are cheaper but easier to exploit. Use layered oversight and sampled audits.

图 12-1 分层监督栈：低成本自动信号覆盖大多数样本，高成本独立监督用于校准和审计。

监督层	比例建议（示意）	用途
Deterministic verifier	尽可能全量	客观正确、格式、执行结果
LLM Judge / RM	大规模	语义质量与软偏好
Critic-assisted human	小规模高价值切片	复杂错误与争议样本
Gold expert	极少但持续	校准、OOD、launch gate

Weak-to-Strong 问题：随着 policy 变强，监督者可能弱于被监督模型。此时“让弱监督直接给答案”并不够，需要 critique、decomposition、tools、debate、formal evidence 等机制把监督能力放大。

13. Judge / Verifier 如何进入不同后训练阶段

Stage	Judge / Verifier 角色	典型输出
SFT data construction	过滤示范质量、拒绝错误 completion	keep/drop + critique
Preference data	构造 chosen/rejected	pair / ranking
Reward Model training	作为 gold 或辅助标签	preference labels
DPO	决定 pair 方向和 margin	chosen/rejected + confidence
Rejection Sampling	从 N 个候选选最优	rank / score
PPO/GRPO/RLVR	在线 reward	scalar / group rewards / gates
Distillation	筛 teacher trajectories	correct high-quality traces
Test-time scaling	best-of-N / search guidance	verifier score
Eval / launch	独立于训练 reward 的验收	slice metrics / regressions

关键原则：训练 grader 与 launch evaluator 必须保持足够独立。若训练和验收共用同一模型、同一 prompt、同一数据分布，很容易把“过拟合裁判”误认为“能力提升”。

14. 从 Scalar Reward 到 Structured Feedback

单一标量很方便优化，却会压缩掉大量诊断信息。生产系统更适合先生成结构化评判，再决定如何聚合成训练信号。

```
{
  "correctness": {"score": 0.0, "evidence": "unit test #4 failed"},
  "format": {"score": 1.0},
  "helpfulness": {"score": 0.7, "judge_confidence": 0.82},
  "safety": {"verdict": "pass"},
  "task_success": false,
  "grader_version": "grader-2026-08-13",
  "spec_version": "behavior-spec-v17"
}
```

结构化字段	作用
score + verdict	训练与筛选
evidence	可审计、可复现
confidence	低置信样本转人工或 ensemble
error_type	Failure Mining
grader_version	版本回归
spec_version	规范追踪
source / oracle	判断依据可追溯

15. Calibration：一个“会打分”的 Judge 不等于一个“可用”的 Judge

指标	回答的问题	注意
Pairwise Accuracy	偏好方向对不对？	对 tie 与难例敏感
ROC-AUC / PR-AUC	二分类区分度如何？	类不平衡时看 PR-AUC
ECE / Brier	置信度是否可信？	阈值路由很依赖 calibration
Kendall / Spearman	排序与人类/Gold 一致吗？	适合多候选排序
FPR/FNR by slice	哪些领域系统性误判？	平均数会掩盖长尾
Swap Consistency	pair 换序是否稳定？	position bias
Prompt Robustness	grader prompt 改写是否稳定？	防模板脆弱性
OOD Generalization	新任务/更难任务还能判断吗？	PRM 特别关键
Hack Gap	proxy reward 与独立 gold 差多少？	检测 reward hacking
Cost / Latency	每百万样本成本？	Judge 也需要系统预算

部署阈值：不要用一个全局 score threshold 覆盖所有任务。按 domain、difficulty、risk、stage 分层校准；高风险样本使用更高阈值或多裁判共识。

16. Reward Hacking：Judge / Verifier 最危险的失效方式

Reward hacking 不是偶发 bug，而是优化系统的结构性结果：只要代理目标和真实目标有差异，足够强的 policy/search 就会倾向于找到差异。

类型	例子	修复
Style hacking	冗长、自信、格式漂亮骗过 Judge	correctness gate + length control
Test hacking	修改测试 / 特判样例	immutable hidden tests
Format hacking	满足标签但无实质推理	accuracy verifier + format 仅作参考
RM exploitation	重复“正确-looking”步骤刷 PRM	reward cap / delta / adversarial set
Judge mimicry	模仿 Judge 偏好风格	cross-family judge + gold audit
Environment shortcut	达到表面状态但未完成任务	secret checks / state invariants
Citation gaming	堆引用但不支持结论	entailment + source verifier

DeepSeek-R1 的经验：其选择 rule-based accuracy/format rewards，并明确担忧 neural RM 在大规模 RL 中被 reward hacking；这说明“更智能的 learned judge”并不总是优于“更窄但可验证的规则”。

研究警告：已有研究发现，推理 RM 在 inference-time reranking 表现很强，并不意味着它作为 RL reward 一定有效；某些 RM 会被 policy 通过重复不必要正确步骤等方式利用。

17. Anti-Hacking 工程栈

1. 把可机械验证的维度全部 verifier 化，减少 learned proxy 的职责范围。
2. 训练 reward 与独立 gold evaluator 分离；Gold 不参与 policy update。
3. 设置 reward cap / clipping / bounded aggregation，避免单一维度无限放大。
4. 使用 hard gates：关键失败直接归零，而不是允许其他软分数补偿。
5. 维护 adversarial grader set：专门包含已知 shortcut、伪装、冗长、错误自信样本。
6. 定期 judge rotation / ensemble，降低 policy 对单个 judge 的适应性。
7. 监控 proxy-gold gap 随 RL step 的变化；出现 divergence 即触发 stop / review。
8. 对高 reward 样本做随机人工 audit；越靠近 reward 上尾越要审计。
9. 记录 grader evidence 与版本，让异常奖励可以回溯。
10. 环境任务使用 hidden tests、不可变 evaluator、独立权限边界。

18. Ensemble：什么时候多个不完美裁判比一个强裁判更好

组合方式	适用	优点	风险
Majority vote	binary verdict	简单稳健	相关错误无法消除
Weighted average	多维 score	易调权重	权重可能变成新 proxy
Lexicographic	硬约束 + 软偏好	不会被风格抵消正确性	规则优先级需明确
Cascade	cheap verifier→expensive judge→human	节省成本	routing 错误
Disagreement routing	裁判不一致转人工	聚焦难例	需要可靠 disagreement metric
Cross-family ensemble	不同模型家族	降低同源偏差	成本增加

推荐架构： Deterministic gates → specialized learned graders → independent reasoning judge → disagreement / high-risk human audit。不要简单平均所有分数。

19. 数据工程：如何训练一个 Judge / Reward Model / PRM

数据组件	内容	Know-How
Seed prompts	真实任务 + synthetic + failures	覆盖 product distribution 与 hard cases
Candidate pairs	质量相近但关键点不同	比“明显好 vs 明显坏”更有信息量
Adversarial negatives	伪引用、冗长、正确答案坏推理	训练抗 shortcut
Gold evidence	tests / refs / proof / expert rationale	提高标签可解释性
Process labels	步骤正确/错误位置	注意难度与领域覆盖
Judge rationales	evaluation CoT / critique	可蒸馏成 generative evaluator
Calibration split	与训练完全隔离	版本发布 gate
OOD split	更难、更长、新领域	检验监督是否可扩展

Self-Taught Evaluator: 研究表明 evaluator 也可以通过 synthetic contrasting outputs + iterative self-improvement 提升，在不使用人工 preference labels 的设置中显著提高 RewardBench 表现。这提示“Judge Factory”本身也可以形成数据飞轮，但仍必须保留独立 gold / human 校准。

20. Judge Prompt Engineering：不是“你是一个严格裁判”

层	推荐内容
Task statement	模型原始任务与上下文
Criteria	3-7 个原子化维度；定义 positive/negative boundary
Priority	正确性 > 约束 > 风格等明确顺序
Reference/evidence	如果有，应单独提供并说明可信度
Decision protocol	先找关键错误，再评分；或逐维评分后聚合
Output schema	JSON: verdict, scores, evidence, confidence, error_type
Bias control	忽略长度/作者/模型名；pair 双向顺序
Uncertainty	允许 unknown / defer_to_human

Know-How：让 Judge 解释“哪条标准被违反、证据在哪里”，通常比只要求“给 1-10 分”更适合工程审计。对训练 reward，可再把结构化结果压缩成标量。

21. Judge / Verifier Service Contract (生产级)

```
grader_request:
  task_id: string
  domain: coding|math|chat|agent|safety
  prompt: ...
  response: ...
  trajectory: optional
  references: optional
  environment_ref: optional
  spec_version: behavior-spec-v17
  grader_plan: auto
```

```
grader_response:
  verdict: pass|fail|tie|unknown
  scores:
    correctness: 0.0..1.0
    task_success: 0.0..1.0
    style: 0.0..1.0
  evidence: [...]
  error_types: [...]
  confidence: 0.0..1.0
  raw_grader_outputs: [...]
  grader_versions: [...]
  latency_ms: ...
  cost_units: ...
```

- 所有 reward 必须可追踪到 grader_version + spec_version。
- 支持 unknown，而不是强迫所有复杂样本二元判断。
- 支持 evidence 和 raw outputs，方便后续 audit。
- 把 routing、aggregation 和 grader model 解耦，以便独立迭代。

22. Coding Agent 案例：构建一个真正可用的 Verifier Stack

阶段	Verifier / Judge	信号
任务读取	repo snapshot hash	环境是否一致
工具调用	schema validator	参数合法
代码修改	diff validator	仅修改允许文件/范围
编译	compiler	compile pass
测试	hidden unit/integration tests	pass ratio
静态分析	lint/type/security analyzer	issue count
任务语义	LLM Judge + task spec	是否满足非测试要求
轨迹质量	critic	是否存在危险 shortcut / 无意义循环
成本	tool/token meter	penalty
最终验收	independent hidden evaluator	launch/eval only

推荐 reward: $R = \text{Gate}(\text{compile}, \text{hidden_tests}, \text{invariants}) \times [\text{task_success} + \text{semantic_score}] - \text{cost_penalty} - \text{shortcut_penalty}$

为什么这么设计: 如果只用 LLM Judge 看 final answer, Agent 可以“说自己完成了”; 如果只用公开 tests, 又可能学会针对测试。环境状态、hidden tests、semantic judge 与轨迹审计要共同组成 verifier stack。

23. Reasoning RL 案例：从 Rule Reward 到 PRM / Generative Verifier

方案	训练信号	优点	风险
Exact answer RLVR	final 0/1	最稳、便宜	过程不可控
Format + accuracy	结构 + 正确	帮助稳定解析	format 可能变成形式主义
ORM	learned outcome score	覆盖不可直接验证任务	proxy hacking
PRM	step rewards	dense credit	昂贵 + OOD
Generative verifier	verify CoT + verdict	可扩 test-time compute	成本高、仍可能自信误判
Formal verifier	machine-check	高置信	覆盖窄

Recipe 选择：如果 final answer 可机械验证，优先从 rule-based outcome reward 起步；只有在它不能提供足够 learning signal 时，才引入 learned process/outcome reward，并用独立可验证成功信号约束。

24. Evaluation：如何评估 Judge / Verifier，而不是只评估 Policy

Eval Suite	必须包含
In-domain	常规任务与已知标签
Near-tie	两个回答都很强，仅有一处关键差异
Adversarial style	错误但很自信/很长/格式漂亮
Order swap	pair A/B 交换
Model-family swap	不同生成模型来源
OOD difficulty	比训练 grader 更难
Long-context	错误跨多段/多文件
Hack set	专门针对已知 reward exploit
Disagreement set	human/judges 不一致样本
Temporal drift	新 policy / 新 spec / 新工具版本

Launch Gate: 任何用于在线 RL 的 grader 更新，都应该先跑 “grader regression suite”；任何 policy 更新，也应该使用独立 evaluator 检查 proxy-gold gap 是否扩大。

25. 18 类常见失败模式

#	失败	后果
1	只用单一 LLM Judge	相关偏差被 policy 学会
2	训练 RM 与 Eval 共用	过拟合被误判为提升
3	只看平均准确率	长尾高风险切片崩溃
4	pair 不做 swap	position bias
5	忽略长度	verbosity bias
6	不保存 evidence	无法审计
7	RM 太久不更新	policy shift 后 stale
8	hard constraint 线性加权	软分数抵消关键失败
9	PRM 只在简单题训练	难题 OOD 失效
10	公开 tests 当唯一 reward	test hacking
11	Judge 与 generator 同源	self-preference / style coupling
12	所有样本强制 binary	把不确定性伪装成确定性
13	只评 reranking 不评 RL	inference 强 $\neq$ training reward 强
14	不做 reward cap	局部代理信号被无限利用
15	grader prompt 无版本	实验不可复现
16	不做对抗样本	shortcut 长期隐藏
17	不计算 grader 成本	评价成为 rollout 瓶颈
18	无人工抽样审计	系统性错误长期积累

26. 成熟度模型：从“一个 Judge Prompt”到“Evaluator Control Plane”

Level	系统形态	特征
L0	Manual / ad hoc	人工看结果，几乎无自动 grader
L1	Single Judge	固定 LLM prompt 或单一 RM
L2	Typed Graders	按任务路由 deterministic / judge / RM
L3	Calibrated Hybrid	ensemble、evidence、slice calibration、gold audit
L4	Adaptive Evaluator Control Plane	动态 hard mining、grader refresh、disagreement routing、anti-hacking loop、独立 launch eval

目标形态： Judge / Verifier 不再是一个模型，而是一个可版本化、可路由、可校准、可审计、能随 policy 演化持续更新的评价控制面。

27. 可直接复用的 Judge / Verifier 设计清单

- ☐ Objective 是否已经拆成可判断的原子 criteria?
- ☐ 哪些维度可以 deterministic / executable / formal verify?
- ☐ 哪些维度确实需要 learned judge?
- ☐ 是否区分 correctness、style、safety、task success?
- ☐ 是否存在 hard gates，而不是所有东西线性加权?
- ☐ Judge 是否做 pair swap / prompt perturbation / length control?
- ☐ 是否有独立 gold / human calibration set?
- ☐ 是否有 OOD + hard + adversarial slices?
- ☐ 是否监控 proxy reward 与 independent eval 的 gap?
- ☐ 是否保存 evidence、confidence、grader_version、spec_version?
- ☐ 在线 RL 的 reward 是否有 cap / bound?
- ☐ 高 reward 上尾是否做随机人工审计?
- ☐ Agent 环境是否有 hidden tests / immutable evaluator?
- ☐ PRM 是否做跨难度 ProcessBench-style error localization eval?
- ☐ Judge 更新与 Policy 更新能否独立回滚?
- ☐ 评价成本是否进入 GPU-hour / rollout budget?

28. 最核心的 Know-Why / Know-How

结论	Know-Why	Know-How
1. 能 verifier 就别只 judge	外部真值更难被 policy 利用	把 correctness 拆成 test/parser/compiler/proof/environment
2. Judge 是代理，不是事实	LLM 评价有系统偏差	swap、rubric、evidence、ensemble、 human calibration
3. Reward 与 Eval 必须独立	否则 Goodhart 不可见	proxy grader + independent gold evaluator
4. Process reward 更密但更脆	step supervision 增强 credit assignment， 也增加攻击面	PRM + outcome gate + OOD process eval
5. Critic 是监督放大器	未来人类可能看不懂强模型输出	AI 找问题 + Human 最终裁决
6. Reward system 必须版本化	行为变化可能来自 grader 而非 policy	spec/grader/data/version lineage
7. 评价也是一个数据飞轮	Policy 变强后旧 grader 会 stale	failure mining → new grader data → recalibrate
8. 最终目标不是最高 Judge 分	真实目标在系统外部	持续测 proxy-gold gap 与真实 task success

Judge / Verifier 的终极问题：

“我们是在训练模型解决真实任务，还是训练模型更会通过我们的评分系统？”

29. 推荐阅读路线与主要研究依据

[1] Zheng et al., 2023 Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

<https://arxiv.org/abs/2306.05685>

LLM Judge 的可用性与 position/verbosity/self-enhancement bias。

[2] Lightman et al., 2023 Let's Verify Step by Step

<https://arxiv.org/abs/2305.20050>

Process supervision vs outcome supervision; PRM800K。

[3] Uesato et al., 2022 Solving math word problems with process- and outcome-based feedback

<https://arxiv.org/abs/2211.14275>

过程监督对 reasoning error 的价值。

[4] Cobbe et al., 2021 Training Verifiers to Solve Math Word Problems

<https://arxiv.org/abs/2110.14168>

生成多个答案 + verifier reranking 的经典范式。

[5] Lambert et al., 2024 RewardBench: Evaluating Reward Models for Language Modeling

<https://arxiv.org/abs/2403.13787>

Reward Model 也需要独立 benchmark。

[6] Wang et al., 2024 Self-Taught Evaluators

<https://arxiv.org/abs/2408.02666>

Synthetic self-improvement evaluator。

[7] Zheng et al., 2024/2025 ProcessBench: Identifying Process Errors in Mathematical Reasoning

<https://arxiv.org/abs/2412.06559>

PRM OOD / hard reasoning 泛化评测。

[8] Khalifa et al., 2025 Process Reward Models That Think

<https://arxiv.org/abs/2504.16828>

Generative long-CoT verifier / ThinkPRM。

[9] McAleese et al., 2024 LLM Critics Help Catch LLM Bugs

<https://arxiv.org/abs/2407.00215>

Critic 与 Human+AI scalable oversight。

[10] OpenAI, 2024 Improving Model Safety Behavior with Rule-Based Rewards

<https://openai.com/index/improving-model-safety-behavior-with-rule-based-rewards/>

Rule-Based Rewards。

[11] OpenAI, 2024 Deliberative Alignment

<https://openai.com/index/deliberative-alignment/>

Spec-aware reward model 与 reasoning over policies。

[12] Bai et al., 2022 Constitutional AI: Harmlessness from AI Feedback

<https://arxiv.org/abs/2212.08073>

AI feedback / preference model / constitution。

[13] DeepSeek-AI, 2025 DeepSeek-R1

<https://arxiv.org/abs/2501.12948>

Rule-based accuracy/format reward 与 neural RM reward hacking 讨论。

[14] Gao, Schulman, Hilton, 2022 Scaling Laws for Reward Model Overoptimization

<https://arxiv.org/abs/2210.10760>

代理 reward 过优化 / Goodhart。

[15] Gao et al., 2024 On Designing Effective RL Reward at Training Time for LLM Reasoning

<https://arxiv.org/abs/2410.15115>

Inference-time RM 强不代表 RL reward 强; reward hacking。

[16] Liu et al., 2023 G-Eval

<https://arxiv.org/abs/2303.16634>

Rubric + CoT 的 LLM evaluation。

[17] Kim et al., 2023 Prometheus

<https://arxiv.org/abs/2310.08491>

可定制 rubric 的 evaluator model。

[18] Burns et al., 2023 Weak-to-Strong Generalization

<https://arxiv.org/abs/2312.09390>

强模型时代的弱监督问题。

[19] OpenAI, 2024 Finding GPT-4's mistakes with GPT-4

<https://openai.com/index/finding-gpt4s-mistakes-with-gpt-4/>

CriticGPT 与人机协同监督。

[20] Anthropic, 2022 Constitutional AI overview

<https://www.anthropic.com/news/constitutional-ai-harmlessness-from-ai-feedback>

原则驱动 AI feedback。

[21] Anthropic, 2026 Next-generation Constitutional Classifiers

<https://www.anthropic.com/research/next-generation-constitutional-classifiers>

运行时 verifier/classifier ensemble；说明 evaluator 架构也趋向多层组合。

30. 一页式工程总结

第一原则：先问“是否有外部可验证事实”，再问“需要哪个 Judge 模型”。

任务性质	首选监督	次级监督	独立验收
确定答案	exact/rule verifier	ORM/critic	held-out exact set
代码	compiler + hidden tests	LLM semantic judge	independent hidden tests
数学推理	final verifier	PRM / generative verifier	hard OOD process benchmark
开放写作	rubric LLM judge	human preference	blind human slice
安全行为	policy-aware/RBR + classifier	human + LLM	red-team / independent eval
Agent	environment success + invariants	trajectory critic	hidden environment eval

Objective → Criteria → Verifiability Decomposition → Grader Routing → Evidence → Calibration → Aggregation → Reward / Pair / Filter → Independent Eval → Failure Mining → Grader Refresh

最终判断：成熟的 Judge / Verifier 系统不是一个“裁判模型”，而是一套持续校准的评价控制面。它的核心 KPI 不是“和某个 benchmark judge 一致”，而是：在 policy 持续变强和主动优化的情况下，训练信号仍然与真实目标保持一致。